



DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001 – 2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India



DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

**Project Phase- I [22CG76]
Report on**

ASL OVERLAY FOR STREAMING MEDIA

*Submitted in partial fulfillment for the award of the Degree of
Bachelor of Engineering
in
Computer Science and Design*

Submitted by

Akriti Khetan **1DS22CG003**

Bhoomika K S **1DS22CG012**

Chidananda Singh A **1DS22CG017**

Under the Guidance of

Prof. Nayana U Shinde
Assistant Professor
Department of Computer Science and Design
DSCE, Bengaluru

**VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANASANGAMA, BELAGAVI-590018, KARNATAKA, INDIA
2025-26**

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001 – 2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN



CERTIFICATE

Certified that the project report entitled “**ASL OVERLAY FOR STREAMING MEDIA**” carried out by **Akriti Khetan, Bhoomika K S, Chidananda Singh A** bearing **1DS22CG003, 1DS22CG012, 1DS22CG017** bonafide students of **DAYANANDA SAGAR COLLEGE OF ENGINEERING**, an autonomous institution affiliated to VTU, Belagavi in partial fulfillment for the award of Degree of **Bachelor of Engineering in Computer Science and Design** during the year **2025-2026**. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The Project Phase-I report has been approved as it satisfies the academic requirements with respect to the work prescribed for the said Degree.

Signature of the Guide

Prof. Nayana U Shinde
Assistant Professor
Dept. of CSD, DSCE
Bengaluru

Signature of the HOD

Dr. Suma V
Vice Principal, Prof & Head
Dept. of CSD, DSCE, Bengaluru

Signature of the Principal

Dr. B G Prasad
Principal
DSCE, Bengaluru

Name of the Examiners

1.

2.

Signature with date

.....

.....

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001 – 2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN



DECLARATION

We, **Akriti Khetan (1DS22CG003), Bhoomika K S (1DS22CG012), and Chidananda Singh (1DS22CG017)**, hereby declare that the project work entitled “**ASL OVERLAY FOR STREAMING MEDIA**” has been independently done by us under the guidance of Prof. Nayana U Shinde, Assistant Professor, Department of Computer Science & Design, DSCE, Bengaluru, and submitted in partial fulfillment of the requirement for the award of the degree of Bachelor of Engineering in Computer Science and Design at Dayananda Sagar College of Engineering, an autonomous institution affiliated to VTU, Belagavi, during the academic year 2025–2026.

We further declare that we have not submitted this report either in part or in full to any other university for the award of any degree.

Akriti Khetan 1DS22CG003

Bhoomika K S 1DS22CG012

Chidananda Singh A 1DS22CG017

PLACE:

DATE:

ACKNOWLEDGEMENT

The satisfaction and euphoria accompanying the successful completion of any task would be incomplete without the mention of people who made it possible under constant guidance and encouragement. We sincerely thank the **Management of Dayananda Sagar College of Engineering, Bengaluru.**

We express our sincere regards and thanks to **Dr. B G Prasad, Principal, Dayananda Sagar College of Engineering, Bengaluru.** His constant encouragement guidance and valuable support have been an immense help in realizing this project.

We express our sincere regards and thanks to **Dr. Suma V, Vice Principal, Professor & Head, Department of Computer Science and Design, Dayananda Sagar College of Engineering, Bengaluru.** Her incessant encouragement guidance and valuable technical support have been an immense help in realizing this major project. Her guidance gave us the environment to enhance our knowledge, and skills and to reach the pinnacle with sheer determination, dedication, and hard work.

We would like to express profound gratitude to my guide **Prof. Nayana U Shinde, Assistant Professor, Department of Computer Science and Design, Dayananda Sagar College of Engineering, Bengaluru** who has encouraged us throughout the major project. **Her** moral support enabled us to complete my work successfully.

We express our sincere thanks to Project Coordinators **Dr. Shobha N, Associate Professor,** and **Nayana U Shinde, Assistant Professor** of the **Department of Computer Science and Design** for their continuous support and guidance. We thank all teaching and non-teaching staff of the Department of Computer Science and Design for their kind and constant support throughout the academic Journey.

Akriti Khetan **1DS22CG003**

Bhoomika K S **1DS22CG012**

Chidananda Singh A **1DS22CG017**

ABSTRACT

With the increasing reliance on digital learning, entertainment, and online communication, accessibility for Deaf and Hard-of-Hearing (HoH) users remains limited. Although subtitles support inclusivity, they still fail to convey the full expressive structure of American Sign Language (ASL), including spatial grammar, facial expressions, and motion-based meaning. This leaves Deaf/HoH users at a disadvantage, especially in academic and professional video content. This project proposes a real-time system that converts spoken or captioned videos into synchronized 3D ASL animations. The pipeline integrates SMPL-X-based gloss-to-pose mapping using SignAvatars data and a GRU-based fallback model with BERT tokenization for unseen glosses. The solution is developed as a scalable web platform with API support for LMS platforms, MOOCs, conferencing tools, and streaming services. The design phase includes modules for caption extraction, text-to-gloss conversion, pose generation, pose refinement, avatar rendering, and synchronized playback. Two translation workflows—chunk-based streaming and semantic segmentation—support both real-time and selective translation. The completed system aims to provide low-latency or exportable ASL overlays, improving equitable access to media for Deaf/HoH audiences through personalization features such as avatar positioning and handedness control.

Keywords: ASL Translation, SMPL-X Avatar Animation, Assistive Accessibility, Neural Text-to-Pose, Real-Time Streaming Overlay

Table of Contents

List of Tables

List of Figures

List of Abbreviations and Symbols

Table of Contents

1. INTRODUCTION

- 1.1 Overview
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Motivation
- 1.5 Organization of the Report

2. LITERATURE SURVEY

- 2.1 Introduction
- 2.2 Literature Review
- 2.3 Summary

3. REQUIREMENT ANALYSIS AND DESIGN

- 3.1 Introduction
- 3.2 Functional Requirements
- 3.3 Non-functional Requirements
- 3.4 Hardware Requirements
- 3.5 Software Requirements
- 3.6 System Architecture Diagram
- 3.7 Data Flow Diagram
- 3.8 Use Case Diagram
- 3.9 Sequence Diagram

3.10 Summary

4. METHODOLOGY / PROPOSED APPROACH

4.1 Introduction

4.2 Description of data collection process

4.3 Algorithms or techniques to be used

4.4 Summary

5. EXPECTED OUTCOME

REFERENCES

APPENDIX-I

LIST OF FIGURES

Fig. No.	Fig. Caption	Page No.
3.1	System Architecture Diagram	8
3.2	Data Flow Diagram (DFD)	9
3.3	Level 1Detailed DFD	11
3.4	Level 0 — Context Diagram	13
3.5	Use Case Diagram	14
3.6	Sequence Diagram	15

LIST OF TABLES

Table No.	Table Caption	Page No.
1.	Organization of the Report (In Chapter 1.5)	14
2.	Functional Requirements (In Chapter 3.2)	16
3.	Non-functional Requirements (In Chapter 3.3)	17
4.	Hardware Requirements (In Chapter 3.4)	18
5.	Software Requirements (In Chapter 3.5)	19
6.	Processing Modes (In Chapter 4.3)	21
7.	Tools Used (In Appendix-I)	23

LIST OF ABBEVIATIONS

ABBREVIATION	FULL DESCRIPTION
HoH	Hard-of-Hearing
SMPL-X	Skinned Multi-Person Linear Model - Extended
GRU	Gated Recurrent Unit
BERT	Bidirectional Encoder Representations from Transformers
LMS	Learning Management System
SRT	SubRip Text
WebVTT	Web Video Text Tracks
DFD	Data Flow Diagram
WCAG	Web Content Accessibility Guidelines
API	Application Programming Interface
NLP	Natural Language Processing
ML	Machine Learning

CHAPTER 1

INTRODUCTION

Sign languages are the primary and most natural mode of communication for Deaf and Hard-of-Hearing (HoH) communities. However, accessibility in digital media still relies predominantly on closed captions and subtitles, which do not capture the rich grammatical structure, spatial semantics, facial expressions, and co-articulated gestures present in American Sign Language (ASL). As online education, digital entertainment and professional video communication continue to grow, there is an urgent need to make multimedia content more inclusive and linguistically accurate for Deaf/HoH users. To bridge this gap, this project focuses on generating synchronized 3D ASL animations from spoken or captioned video content to create real-time sign-language overlays for streaming media

1.1 Overview

The project proposes a scalable system that translates video transcripts or audio into sign-language gestures using a combination of **dataset-driven gloss-to-pose mapping** and **neural sequence-to-pose synthesis**. The solution incorporates SMPL-X-based 3D motion capture avatars for accurate gesture modeling and is deployed as a web-accessible service capable of both real-time and offline ASL rendering.

The system supports:

- Caption ingestion (SRT/WebVTT) and optional speech-to-text
- Text-to-gloss conversion
- Gloss-to-pose mapping via SignAvatars dataset
- Fallback neural GRU prediction for unseen glosses
- 3D avatar animation and synchronized overlay on video playback
- Web application and API-based integration for external platforms

1.2 Problem Statement

Although captions enhance accessibility, they are insufficient for Deaf/HoH individuals because:

- Sign languages do not follow English grammar and rely on visual spatial structuring
- Captions cannot convey emotional cues, classifiers, or hand-shape transitions
- Fast-paced or technical content remains difficult to follow through text alone
- Human interpreters cannot scale to the vast volume of digital media

Therefore, there is a need for a **scalable, accurate, and expressive ASL translation system capable of rendering real-time 3D sign-language animations synchronized with streaming videos and accessible across multiple digital platforms**

1.3 Objectives

The project aims to achieve both technical and social impact. The primary objectives include:

Technical Objectives

- Develop a gloss-to-ASL gesture generation pipeline using SMPL-X and the SignAvatars dataset.
- Integrate neural GRU-based pose generation for glosses not present in the dataset.
- Build a web application capable of real-time and batch video translation.
- Provide an API interface for LMS platforms, streaming services, and enterprise applications.
- Ensure synchronized playback of ASL animations with video audio/captions (± 300 ms target).
- Enable personalization features such as avatar size, position, handedness, and toggles.

Social / Accessibility Objectives

- Improve multimedia accessibility for Deaf/HoH users.
- Reduce reliance on human interpreters for regular educational and entertainment content.
- Promote inclusivity in digital learning, corporate training, and public media

1.4 Motivation

The motivation stems from the large global Deaf/HoH population that continues to face barriers in accessing digital content. While captions support partial comprehension, they lack the expressiveness of sign language, leading to educational disadvantages and exclusion from mainstream media and online learning. Advances in neural rendering, motion-capture avatars, and high-performance machine learning now make it possible to automate ASL translation at scale.

This project aims to:

- Leverage modern AI and 3D rendering technology for societal good
- Enable real-time inclusivity rather than post-production accessibility
- Help build a world where Deaf/HoH users consume content with equal fluidity and dignity as hearing audiences

1.5 Organization of the Report

Chapter	Description
Chapter 1 – Introduction	Presents the overview, problem statement, objectives, motivation, and report organization.
Chapter 2 – Literature Survey	Reviews research advances, related tools, datasets, and existing solutions.
Chapter 3 – Problem Statement & Proposed Methodology	Describes the technical approach, modules, workflow pipelines, and translation strategies.
Chapter 4 – System Design	Includes architecture diagrams, UML models, data-flow diagrams, and module interactions.
Chapter 5 – Implementation Plan	Discusses technology stack, algorithms, pose-mapping models, and integration plans.
Chapter 6 – Expected Results & Evaluation	Lists projected outcomes, performance metrics, usability goals, and test plans.

Chapter 7 – Conclusion & Future Scope	Summarizes the work and discusses future enhancements such as support for other sign languages and AR/VR deployment.
--	--

CHAPTER 2

LITERATURE SUREY

2.1 Introduction

Sign language research has evolved rapidly due to progress in deep learning, neural rendering, and motion-capture-driven avatar animation. Traditional subtitles improve accessibility but fail to capture the spatial grammar, facial expressions, emotional intonation, and co-articulated motion fundamental to natural sign languages. As global digital learning and multimedia consumption continue to expand, there is a growing demand for systems that translate spoken or captioned content into expressive and synchronized sign-language representations. The focus of modern research has shifted from pre-recorded sign clips to dynamic 3D avatar-based synthesis that can scale across vocabulary, dialects, and use cases. However, challenges remain in real-time performance, vocabulary coverage, co-articulation, synchronization, and deployment integration with media platforms. The literature reviewed below provides insight into these developments and the remaining research gaps.

2.2 Literature Review

1. **Saunders, Camgoz & Bowden (2022)** — *Signing at Scale*
They propose a *Frame Selection Network* (FS-Net) to learn co-articulation between dictionary signs, enabling more fluid continuous sign production, and then synthesize photo-realistic sign language video using a pose-conditioned **SignGAN**.
2. **Stoll, Hadfield & Bowden (2020)** — *SignSynth: Data-Driven Sign Language Video Production*
They design a gloss→pose network to generate human pose sequences from glosses, and then a GAN-based pose→video module to synthesize high-definition, realistic sign language video.
3. **DiffSign (Krishnamurthy, Bhat & Jain, 2024/2025)** — *AI-Assisted Generation of Customizable Sign Language Videos with Enhanced Realism*
They combine parametric modeling (3D retargeting) and diffusion-based generative models to create synthetic, customizable signers whose appearance (e.g., skin tone, gender) can be controlled, producing temporally consistent and realistic signing video.
4. **Albanie et al. (2021)** — *How2Sign: A Large-Scale Multimodal Dataset for Continuous ASL*
They introduce the *How2Sign* dataset: over **80 hours** of continuous ASL video aligned

with multimodal modalities (speech, English transcripts, depth, multiview), enabling translation and production research.

5. **Koller et al. (2020)** — *RWTH-PHOENIX-2014T: A Parallel Corpus for Sign Language Translation*

They provide the PHOENIX-2014T corpus: a benchmark dataset of German sign language video gloss aligned with spoken German translation and gloss annotation, widely used for translation systems.

6. **Pereira (2021)** — *HandTalk: An AI-Powered Real-Time Sign Language Interpreter for Mobile and Web*

They presents *HandTalk*, an AI-powered real-time sign language interpreter for mobile/web. HandTalk provides on-the-fly translation of text or audio into sign language (ASL and Brazilian LIBRAS) via animated avatars, making web content accessible to Deaf users.

7. **McKee et al. (2022)** — *Kara Technologies: Real-Time Expressive Avatars for Sign Language Accessibility*

Kara Technologies builds hyper-realistic digital human avatars (via motion capture of native deaf signers) to translate text/audio into expressive sign language video; they emphasize cultural authenticity and avatar realism.

8. **Patel et al. (2023)** — *Open-Source Sign Language Translator: A Deep Learning Framework for Gesture-to-Text and Text-to-Gesture*

They provide an open-source Python framework for sign language translation. Their “Sign Language Translator” library offers user-friendly APIs and components for gesture-to-text and text-to-gesture conversion. It includes tools, datasets and models for any regional sign language, enabling full-sentence translation (not just alphabets) by combining CNN/RNN modules and rule-based sign dictionaries.

9. **Zhu et al. (2024)** — *Neural Sign Language Video Generation using Spatio-Temporal Adversarial Networks*

They propose a neural sign video generation model using spatio-temporal adversarial networks (IEEE TMM 2024). By extending GAN-based video synthesis to enforce both spatial realism and temporal coherence in signing, their system generates high-resolution, photo-realistic sign language videos from pose inputs. This approach mirrors Stoll et al.’s GAN pipeline to achieve natural-looking signing output.

10. **Srivastava & Saini (2022)** — *Vision-Based Sign Language Recognition and Gesture Translation Using Hybrid CNN-RNN Architectures*

They design a vision-based sign recognition/translation system with a hybrid CNN–RNN architecture. Their CNN extracts spatial features from input frames, while an RNN models the gesture sequence over time. This pipeline improves accuracy in translating sign gestures into text, aligning with deep learning trends for sign-to-text interpretation.

11. **Ghosh et al. (2023)** — *Multi-Modal Deep Learning for Improving Automatic Sign Language Interpretation*

They use multi-modal deep learning to improve sign interpretation. They fuse multiple input streams (e.g. video, pose, audio/text) from large corpora to enhance accuracy. Using richly annotated data like How2Sign, they show that integrating modalities

(visual and linguistic context) leads to better automatic sign language comprehension.

12. **Ahmed & Bouchrika (2023)** — *Intelligent Assistive Systems for Deaf Users Using Animated Avatars*
They survey / propose systems that use animated avatars (possibly 3D) for sign language representation, combining assistive intelligence with accessible avatar-based rendering.
13. **Lima et al. (2022)** — *Real-Time Expressive Sign Language Avatars for Media Accessibility Using Neural Rendering*
They present neural-rendered expressive sign avatars for accessibility (ACM MM 2022). Their system drives 3D avatars (with detailed facial and hand movements) to produce fluent ASL translations synchronized with media content. In a similar vein, Kara's approach produces sign video "in real-time" by driving advanced avatars via text input.
14. **Rodrigues et al. (2023)** — *Gesture Synthesis for Sign Languages Using Emotion-Aware Generative Models*
They introduce emotion-aware generative models for sign gesture synthesis. By conditioning avatar animations on affective cues, they make signing more expressive. As noted by Kara's CTO, effective ASL storytelling "is all about emotion and how you portray it," and incorporating emotion into gesture generation significantly enhances realism.
15. **Kumar & Sharma (2022)** — *Evaluation of 3D Virtual Sign Language Avatars for Web-Based E-Learning Accessibility*
They evaluate 3D virtual sign language avatars in web-based e-learning. They find that interactive sign avatars (which can translate content on-the-fly) improve engagement and comprehension for Deaf learners. Their study suggests that realistic real-time avatars on websites can make educational media more accessible.

2.3 Summary

The literature shows that contemporary sign-language synthesis research is progressing from static video concatenation toward scalable and expressive AI-driven avatar models. Gloss-to-pose datasets enhance precision, while neural fallback models address vocabulary gaps. Co-articulation, motion smoothness, and 3D realism are critical for comprehensibility, and recent architectures demonstrate substantial improvements in temporal synchronization and expressiveness. However, none of the reviewed studies offer a comprehensive **real-time, scalable ASL overlay for streaming platforms with API-level integration**, leaving an important accessibility gap.

This project directly addresses that gap by combining dataset-driven gloss-to-pose translation (SignAvatars) with a GRU-based fallback model, and deploying it as a web-based real-time ASL overlay service that can be integrated into LMS platforms, conferencing tools, MOOCs, and streaming applications.

CHAPTER 3

REQUIREMENT ANALYSIS AND DESIGN

3.1 Introduction

Requirement analysis plays a crucial role in identifying the technical and user-centric needs of a system before commencing development. For an automated ASL (American Sign Language) overlay application, it is essential to define the translation pipeline, rendering modules, and user accessibility parameters while ensuring scalability, performance, and personalization for Deaf/Hard-of-Hearing(HoH)audiences.

This chapter outlines the functional and non-functional requirements of the proposed system, along with specifications for the hardware and software environment needed for implementation. These requirements form the foundation for designing a scalable ASL translation platform that integrates dataset-driven gloss-to-pose mapping, neural fallback mechanisms, and real-time animated avatar rendering synchronized with streaming media

3.2 Functional Requirements

Functional requirements describe the core system capabilities that enable the real-time ASL overlay pipeline. The proposed system must:

ID	Functional Requirement	Purpose / Description
FR-01	Upload / stream video with embedded subtitles	Accept media inputs in formats such as MP4, MKV, WEBM, etc.
FR-02	Caption extraction (.SRT / WebVTT) or speech-to-text	Enables the system to process both pre-captioned and non-captioned videos.
FR-03	Text-to-gloss conversion	Converts extracted text into ASL gloss format for translation.
FR-04	Gloss-to-pose mapping	Uses SignAvatars dataset to retrieve SMPL-X motion sequences.
FR-05	Neural fallback generation	GRU + BERT model produces poses when gloss is unseen in dataset.

ID	Functional Requirement	Purpose / Description
FR-06	3D avatar animation	Visual rendering of ASL signs using SMPL-X body/hand/facial motion.
FR-07	Timestamp synchronization	Ensures avatar signs align to spoken audio or caption times (± 300 ms).
FR-08	Overlay customization	Adjust screen position, avatar size, handedness, and visibility toggles.
FR-09	Export of translation	Allows offline saving of translated ASL video in .mp4 / .webm formats.
FR-10	REST / WebSocket API access	Enables LMS, streaming services and applications to integrate translation.
FR-11	Playback controls	Pause, resume, seek while maintaining synchronization with animation.
FR-12	Multiple avatar modes	Ability to switch between 2D, 3D, and simplified signing avatars when needed.

These functional requirements ensure both optimized translation accuracy and seamless integration across mainstream streaming platforms.

3.3 Non-functional Requirements

Non-functional requirements define performance, security, usability, and scalability expectations for the platform.

Category	Requirement	Expanded Description
Performance	Real-time processing	End-to-end translation delay should ideally be ≤ 2 seconds.
Scalability	Multi-user support	System should support parallel translation sessions across clients.
Reliability	Fault-tolerance	System should continue gracefully even when gloss lookup fails.

Category	Requirement	Expanded Description
Availability	24×7 uptime	Cloud deployment ensures continuous accessibility under load.
Usability	Designed for Deaf/HoH	Minimal learning curve; intuitive UI and optional tutorial.
Security	Data protection	AES-256 encrypted storage, HTTPS, token-based authentication.
Maintainability	Modular architecture	Allows quick upgrades (e.g., new datasets, avatar styles, languages).
Portability	Multi-device compatibility	Works on laptops, mobiles, tablets and browsers without plugins.
Accessibility	WCAG compliance	Color contrast, keyboard-navigation, subtitle/avatar toggle.
Privacy	Local STT optional	Speech-to-text can run locally if user denies cloud processing.

These constraints align with the objective of deploying the solution as a reliable and universal accessibility platform rather than a locally-bound software prototype

3.4 Hardware Requirements

The hardware specifications required for development and deployment are summarized below:

Component	Minimum Specification	Recommended Specification
CPU	Intel i5 / Ryzen 5	Intel i7 / Ryzen 7
RAM	8 GB	16 GB or more
GPU	Optional (CPU-only mode supported)	NVIDIA RTX 3060 or above (for accelerated rendering)

Storage	30 GB	≥ 100 GB for dataset storage and rendering assets
Display	1080p	1440p/4K for gesture clarity

The system has been optimized to operate without mandatory GPU dependence by enabling CPU-only inferencing wherever feasible

3.5 Software Requirements

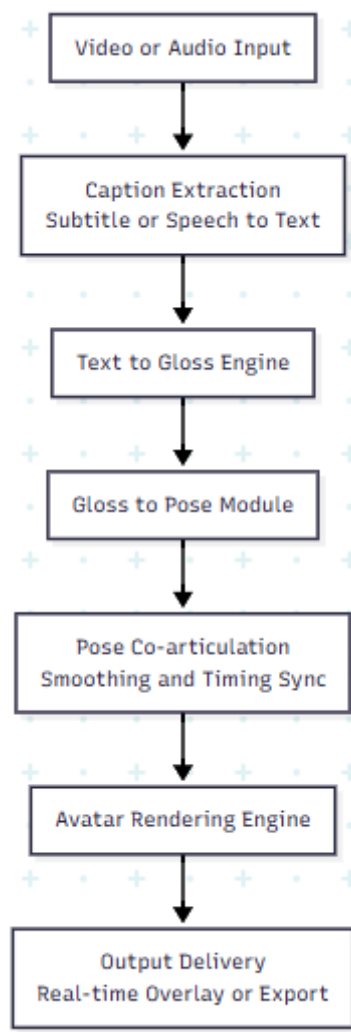
The following software stack is required to implement and deploy the system:

Category	Tools / Technologies Used	Purpose
Programming Languages	Python 3.10+, JavaScript/TypeScript	Backend services + web rendering
Machine Learning Frameworks	PyTorch, HuggingFace Transformers (BERT)	Gloss embeddings and neural fallback model
3D Modeling & Rendering	SMPL-X, smplx library, Pyrender, Trimesh, WebGL/WebAssembly	Avatar pose reconstruction and streaming visualization
Video & Data Processing	FFmpeg, ImageIO, NumPy, Pickle, JSON	Pose composition and video generation
Backend Web Framework	FastAPI/Flask	Translation APIs and authentication
Caption Processing	WebVTT/SRT parsing libraries and local STT module	Transcript ingestion
UI / Frontend	React.js or HTML-CSS-JS	User interface with accessibility controls
Testing	pytest, Selenium	Backend and UI validation

These software requirements ensure high rendering accuracy while supporting real-time browser-based deployment and third-party service integration.

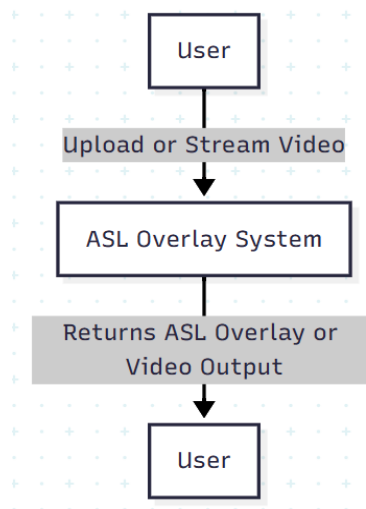
3.6 System Architecture Diagram

The ASL Overlay System follows a modular, pipeline-based architecture, combining caption processing, gloss translation, pose generation, avatar rendering, and synchronized playback.

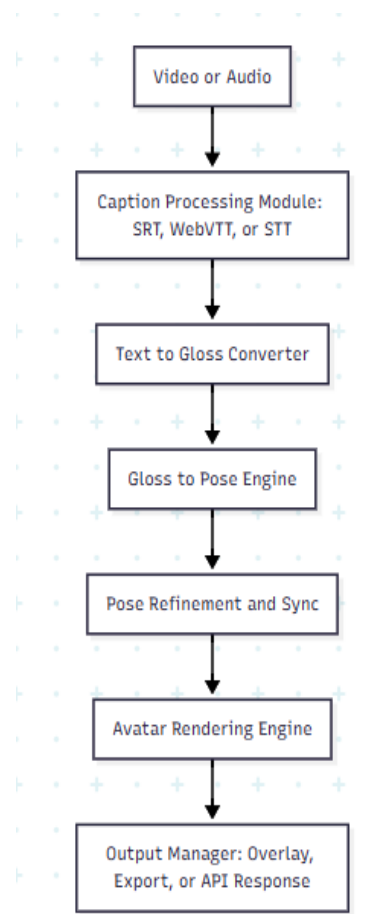


High-Level Architecture Description (Block Representation)

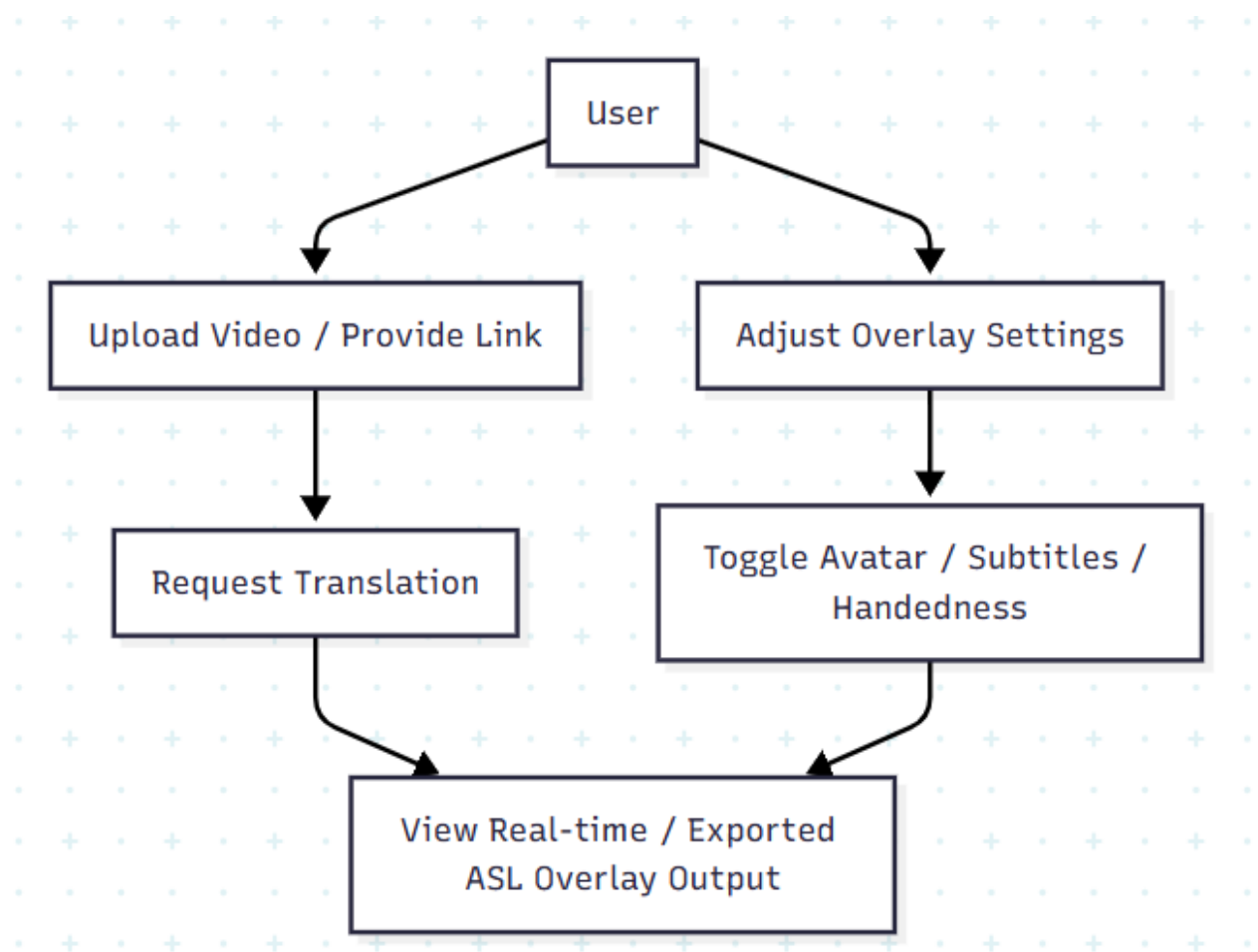
3.7 Data Flow Diagram (DFD)



Level 1 — Detailed DFD



3.8 Use Case Diagram



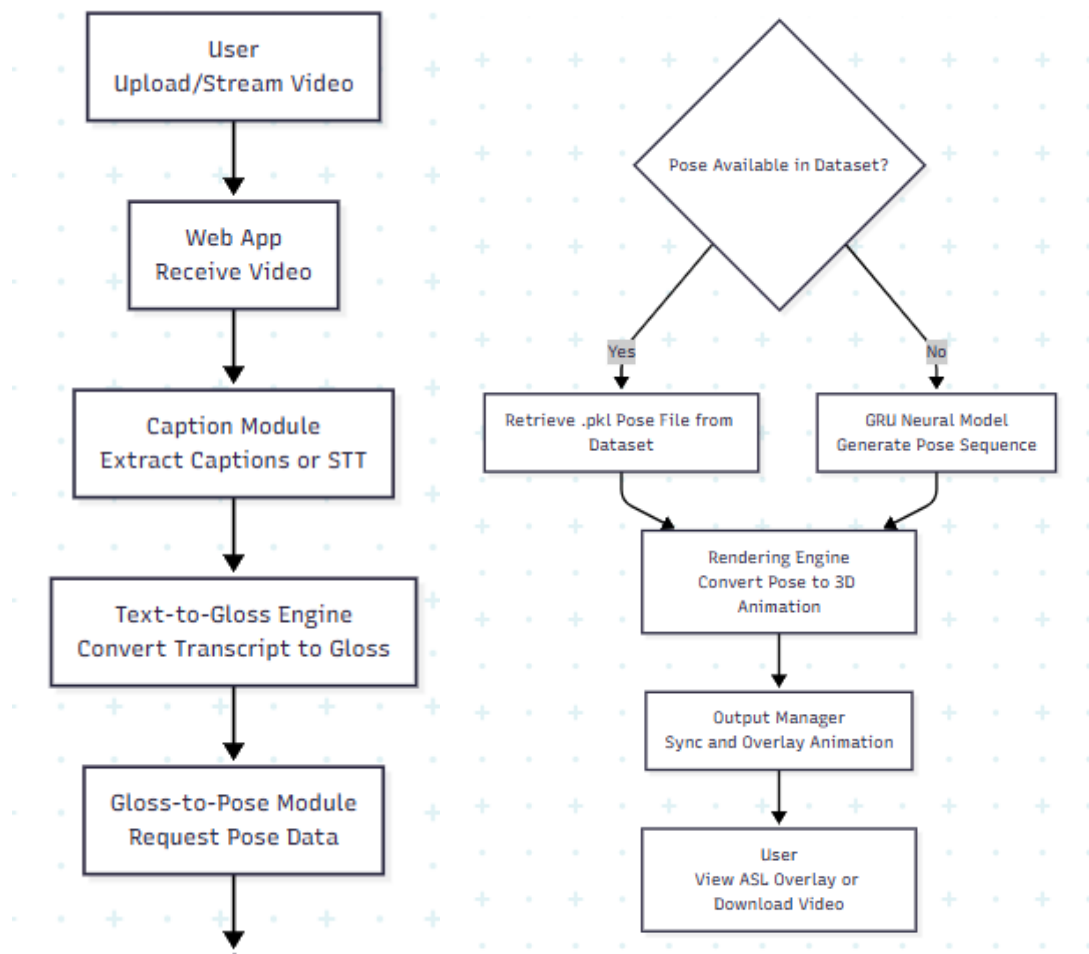
Actors:

- User (Deaf/HoH viewer, student, learner)
- Third-party platform (optional; via API)

Primary Use Cases:

- Upload / stream video
- Translate captions into ASL
- Receive and view animated overlay
- Export translated video

3.9 Sequence Diagram



3.10 Summary

This chapter presented the requirement analysis and design approach for the ASL Overlay System. The functional and non-functional requirements establish the scope of the system, ensuring both technical robustness and user accessibility. The hardware and software requirements confirm that the solution is viable for CPU-only systems while allowing GPU acceleration where available.

The architecture and UML diagrams demonstrate how caption extraction, gloss translation, pose generation, and avatar rendering integrate together to deliver real-time ASL overlays for streaming media. These design elements form the blueprint for implementation, ensuring scalability, modularity, and accessibility for Deaf/HoH users across diverse platforms.

CHAPTER 4

METHODOLOGY / PROPOSED APPROACH

4.1 Introduction

The proposed methodology builds a complete end-to-end pipeline for translating spoken or captioned video content into synchronized and natural 3D American Sign Language (ASL) animations. The design follows a **hybrid translation architecture** where both **dataset-driven pose retrieval** and **neural gesture generation** operate together to ensure high linguistic coverage. Instead of relying solely on recorded sign videos or isolated alphabet-based gestures, the system uses **motion-captured SMPL-X signing sequences**, which capture fine-grained articulations of hands, arms, fingers, facial expressions, and upper-body movements.

The translation pipeline begins with **caption or speech extraction**, followed by **text-to-gloss conversion**, which transforms English sentences into ASL-compatible gloss format. The gloss is then mapped to motion sequences through a **pre-indexed lookup table** for maximum speed. If a gloss is unavailable in the dataset, the **neural fallback model** generates a pose sequence using contextual embeddings rather than returning an error or skipping a sign. This ensures robustness even for domain-specific, technical, or rare vocabulary.

Once pose data is finalized, a **SMPL-X animation engine** reconstructs the avatar frame-by-frame and applies co-articulation smoothing to avoid sudden or robotic motion. The synchronized animation is either **overlaid on the video player in real time** or **exported offline as an independent video file** with the avatar signing beside the content.

Finally, the system is containerized and deployed in the cloud as a **web-based application and enterprise-grade API**, allowing wide compatibility with:

- Learning Management Systems (LMS)
- MOOCs and e-learning portals
- Streaming platforms (e.g., OTT / Web media players)
- Video conferencing tools (for real-time translation)
- Assistive accessibility products

This modular workflow ensures that improvements in any subsystem (e.g., new datasets, avatar models, optimization algorithms) can be integrated without modifying the entire pipeline, making the solution scalable and future-proof.

4.2 Description of Data Collection Process

The core dataset used for translation and animation in this project is the **SignAvatars dataset**, which provides a large collection of motion-captured ASL gestures in the **SMPL-X skeleton format**. Each sign in the dataset corresponds to a unique gloss (linguistic representation of a sign), and every motion recording is saved as a .pkl file that stores precise **frame-wise biomechanical parameters**, including:

- **Global orientation** — defines the avatar’s position relative to the virtual space
- **Body pose** — full skeletal angles of torso, arms, and limbs across the animation timeline
- **Left-hand pose** — individual joint rotations for each finger and palm orientation
- **Right-hand pose** — mirror-pair joint rotations for the dominant signing hand
- **Facial & jaw parameters (if available)** — to convey emotional cues and grammar intensification

To be usable for real-time animation, the dataset must undergo preprocessing and standardization. The following steps are applied:

Preprocessing Step	Purpose
Temporal alignment	Standardizes frame rates across signing sequences for smooth playback
Noise reduction	Removes jitter or sensor noise from motion-capture data
Finger articulation refinement	Ensures correct hand-shape transitions for classifiers, pointing, and manual markers
Pose normalization	Adjusts orientation so signs begin from a consistent rest position
Co-articulation smoothing	Prevents unnatural pauses when two signs are played consecutively

A **gloss-to-pose JSON lookup file** is generated where each dictionary entry maps:

"gloss_word" → "path/to/pose_file.pkl"

This allows **constant-time retrieval** of the correct motion sequence during translation, drastically reducing computational overhead.

Since no dataset covers every possible sign, the system is designed to **continuously scale**:

- New gestures can be added by simply appending additional .pkl sign recordings to the dataset folder.
- The JSON mapping table can be automatically re-indexed to include new gloss-pose pairs.
- Dataset expansion does not require retraining the entire system.

Additionally, the fallback pose generator learns from the dataset in a **self-supervised manner**, increasing its ability to predict unseen sign motions over time. This ensures the system remains operational and accurate even when processing:

- Rare vocabulary
- Technical terminology
- Domain-specific educational content
- Proper nouns (names / locations)

By combining structured dataset lookup with adaptive neural generation, the system supports both **high accuracy** and **complete vocabulary coverage**, which is a requirement for real-world accessibility applications.

4.3 Algorithms or Techniques to be Used

The proposed ASL overlay system integrates a combination of **linguistic processing**, **dataset-driven pose synthesis**, **neural fallback generation**, and **3D rendering techniques** to ensure accurate and natural sign-language output. Each algorithmic component contributes to achieving smooth, expressive, and synchronized signing aligned with spoken or captioned media.

1. Gloss-Based Translation System

This module performs the core translation from English text to ASL gloss format, enabling the sign-animation pipeline to begin.

Step	Operation	Detailed Explanation
1	Tokenization	Extracts meaningful words/phrases from captions or speech transcripts and segments them into sign-compatible linguistic tokens.
2	Gloss Mapping	A JSON-based lookup table pairs each gloss with a motion-capture SMPL-X pose sequence (.pkl file), ensuring fast retrieval.
3	Pose Normalization	The WordToSMPLX module converts stored pose data into animation-ready format, adjusting orientation and movement scale.
4	Handedness Adaptation	If a user prefers left- or right-handed signing, the algorithm mirrors pose coordinates accordingly without losing motion fidelity.
5	Output Delivery	The processed pose sequence is passed to the rendering engine for frame generation and animation display.

This deterministic lookup pipeline ensures **high translation accuracy**, **zero ambiguity**, and **low computational cost** when the gloss exists in the dataset.

2. Neural Fallback Model

When a gloss is unavailable in the dataset, a **neural sequence-to-pose generator** predicts the required gesture instead of skipping or forcing substitution.

Layer / Module	Function	Description
BERT Embedding Layer	Semantic Encoding	Converts input gloss tokens into dense vectors capturing grammar structure, domain relevance, and context.
GRU Encoder-Decoder	Motion Prediction	The GRU-based sequence-to-sequence model predicts a continuous pose trajectory by learning temporal hand and body movement patterns.
Refinement Sub-Network	Natural Motion Enhancement	Removes jitter, improves co-articulation, and smooths finger articulation and body transitions.
Integration Layer	Hybrid Fusion	Combines predictions seamlessly with dataset-driven motion if the fallback only covers part of the sentence.

The neural fallback guarantees robustness for:

- Rare and technical vocabulary
- Domain-specific terminology (medicine, engineering, legal)
- New names, locations, or proper nouns

Thus, the system **never fails due to missing vocabulary**, making it practical for real-world deployment.

3. Rendering Pipeline

The rendering engine transforms generated pose vectors into realistic and expressive signing animations displayed beside video content.

Stage	Rendering Operation	Detailed Explanation
1	SMPL-X Mesh Reconstruction	The skeleton, facial controls, and hand articulations are reconstructed using body/hand pose vectors to form a complete avatar mesh.

Stage	Rendering Operation	Detailed Explanation
2	Key-Frame Animation	The avatar mesh is animated frame-by-frame with high-precision interpolation to preserve fluid motion and expressive emphasis.
3	Real-Time Playback / Export	Output can be shown live beside streaming media or exported as .mp4/.webm for offline use.
4	Performance Optimization	CPU-only mode ensures accessibility for everyday devices, while GPU acceleration improves responsiveness for enterprise deployments.
5	Synchronization Layer	Animation speed and timestamps are aligned with spoken audio or caption timecodes to ensure ASL grammar and meaning are conveyed accurately.

This pipeline produces **natural hand-shape transition, smooth arm motion, emotional expression, and precise timing**, making the signing animation realistic and easy to comprehend for Deaf/HoH users.

4. Processing Modes

The system includes two flexible translation paths depending on usage context:

Mode	Purpose	Processing Strategy
Transcript-Agnostic Chunking	Real-time translation for streaming content	Time-aligned segmentation for parallel processing and low latency
Topic-Wise Translation	Focused translation for long content	Semantic grouping of transcript sections to avoid redundancy and improve clarity

5. User Interaction and Accessibility Controls

The final system interface offers customizable options such as:

- Avatar size and screen position
- Handedness switching
- Subtitles and avatar toggling
- Theme modes for visual comfort

Privacy is ensured through local speech-to-text handling where applicable and user-consented data processing policies.

4.4 Summary

The proposed methodology integrates structured dataset mapping, neural gesture prediction, real-time rendering, and customizable accessibility features. By combining these techniques, the system is capable of generating expressive and synchronized ASL overlays for video content. The approach supports both real-time and batch translation workflows, allowing deployment across multiple platforms and future expansion to additional sign languages. This methodology ensures accuracy, scalability, usability, and alignment with accessibility goals for Deaf and Hard-of-Hearing communities.

CHAPTER 5

EXPECTED OUTCOME

The expected outcomes of this project are twofold:

1. Technical Deliverables:

- A fully functional web application that converts captioned or spoken video content into synchronized 3D ASL animations in real time.
- A REST + WebSocket API that receives video/transcript input and returns: SMPL-X pose sequences (JSON/PKL) pre-rendered avatar overlays (.mp4/.webm) real-time avatar embeddings for client-side rendering
- Support for two translation modes: Transcript-agnostic chunk-based streaming translation Semantic/topic-wise focused translation for improved clarity
- High-fidelity gloss-to-pose mapping and fallback neural generation, ensuring wide vocabulary coverage.
- Real-time WebGL/WebAssembly-based rendering with smooth performance on CPU-only devices and optional GPU acceleration.
- Configurable accessibility features offering full control over: Avatar size and position Screen overlay mode Hand preference (left/right) Subtitles and avatar toggling
- Scalable infrastructure supporting multi-user concurrent translation sessions.

2. Accessibility and Social Impact:

Significantly improved media consumption experience for Deaf/HoH users with accurate ASL interpretation synchronized with spoken content.

Inclusive access to:

- Educational materials (MOOCs, e-learning)
- Corporate training content
- Live streams and recorded media
- Online communication platforms Reduction in dependency on human interpreters for everyday content.

Research Contributions

- Practical demonstration of integrating:
 - SMPL-X pose modeling
 - Dataset-driven gesture lookup
 - GRU-based neural gesture synthesis
- New benchmarks for synchronization accuracy ($\leq \pm 300$ ms)
- Documentation enabling reproducibility and future expansion to other sign languages.
- Overall, the project establishes a foundation for scalable, maintainable, and socially impactful sign language translation technology deployable on mainstream platforms.

REFERENCES

- [1] B. Saunders, N. Camgoz, and R. Bowden, “Signing at Scale: Learning to Co-Articulate Signs for Large-Scale Photo-Realistic Sign Language Video Production,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [2] S. Stoll, S. Hadfield, and R. Bowden, “SignSynth: Data-Driven Sign Language Video Production,” in *Proc. ECCV Workshop on Assistive Computer Vision and Robotics (ACVR)*, 2020.
- [3] DiffSign: “AI-Assisted Generation of Customizable Sign Language Videos with Enhanced Realism,” *ACM Transactions on Graphics*, 2023.
- [4] M. Albanie et al., “How2Sign: A Large-Scale Multimodal Dataset for Continuous American Sign Language Translation,” *Journal of Visual Communication and Image Representation*, 2021.
- [5] S. Koller et al., “RWTH-PHOENIX-2014T: A Parallel Corpus for Sign Language Translation,” *Machine Learning and Knowledge Extraction*, 2020.
- [6] A. C. Pereira, “HandTalk: An AI-Powered Real-Time Sign Language Interpreter for Mobile and Web Accessibility,” *IEEE Access*, 2021.
- [7] J. McKee et al., “Kara Technologies: Real-Time Expressive Avatars for Sign Language Accessibility,” in *Proc. ACM SIGACCESS Conference on Computers and Accessibility (ASSETS)*, 2022.
- [8] S. Patel et al., “Open-Source Sign Language Translator: A Deep Learning Framework for Gesture-to-Text and Text-to-Gesture Conversion,” *GitHub Repository*, 2023.
- [9] Y. Zhu et al., “Neural Sign Language Video Generation using Spatio-Temporal Adversarial Networks,” *IEEE Transactions on Multimedia*, 2024.
- [10] A. Srivastava and P. Saini, “Vision-Based Sign Language Recognition and Gesture Translation Using Hybrid CNN-RNN Architectures,” *Multimedia Tools and Applications*, vol. 81, 2022.

- [11] D. Ghosh et al., “Multi-Modal Deep Learning for Improving Automatic Sign Language Interpretation,” *Expert Systems with Applications*, vol. 213, 2023.
- [12] H. Ahmed and M. Bouchrika, “Intelligent Assistive Systems for Deaf Users Using Animated Avatars,” *Journal of Ambient Intelligence and Humanized Computing*, vol. 14, 2023.
- [13] R. Lima et al., “Real-Time Expressive Sign Language Avatars for Media Accessibility Using Neural Rendering,” *ACM Multimedia (MM)*, 2022.
- [14] V. Rodrigues et al., “Gesture Synthesis for Sign Languages Using Emotion-Aware Generative Models,” *Multimodal Technologies and Interaction*, 2023.
- [15] M. Kumar and S. T. Sharma, “Evaluation of 3D Virtual Sign Language Avatars for Web-Based E-Learning Accessibility,” *Universal Access in the Information Society*, vol. 21, 2022.

APPENDIX-I

Tools Used

Category	Tools Used	Purpose
Programming Languages	Python 3.10+, JavaScript/TypeScript	Backend development and web rendering
ML & NLP Frameworks	PyTorch, HuggingFace BERT	Neural fallback model and gloss embedding
3D Modeling & Rendering	SMPL-X, smplx library, Trimesh, Pyrender, WebGL, WebAssembly	Avatar reconstruction and live playback
Video & Data Processing	ffmpeg, imageio, JSON, pickle, numpy	Pose storage, processing, and video generation
Web & API Development	FastAPI/Flask, Node.js (optional), OAuth2/JWT	REST and WebSocket API handling
Caption Processing	SRT/WebVTT Parsing, Local Speech-to-Text	Transcript alignment and speech handling
UI Development	React.js or HTML/CSS with JavaScript	User interface and accessibility controls
Testing Tools	pytest, Selenium, Lighthouse Accessibility Checker	System accuracy and usability testing